Appointment Capacity Engine — Technical Spec

ML models, capacity math, control loop, implementation steps, corner cases, and end-to-end flow.

1. Purpose & scope

The engine converts a daily lead CSV into fully-booked agent calendars with minimum wasted leads. It replaces "blast every lead" with a closed-loop controller that releases outreach in proportion to real, per-state appointment capacity, and uses ML models to size and prioritize each release. Same-day booking is the default; future-day is a gated fallback.

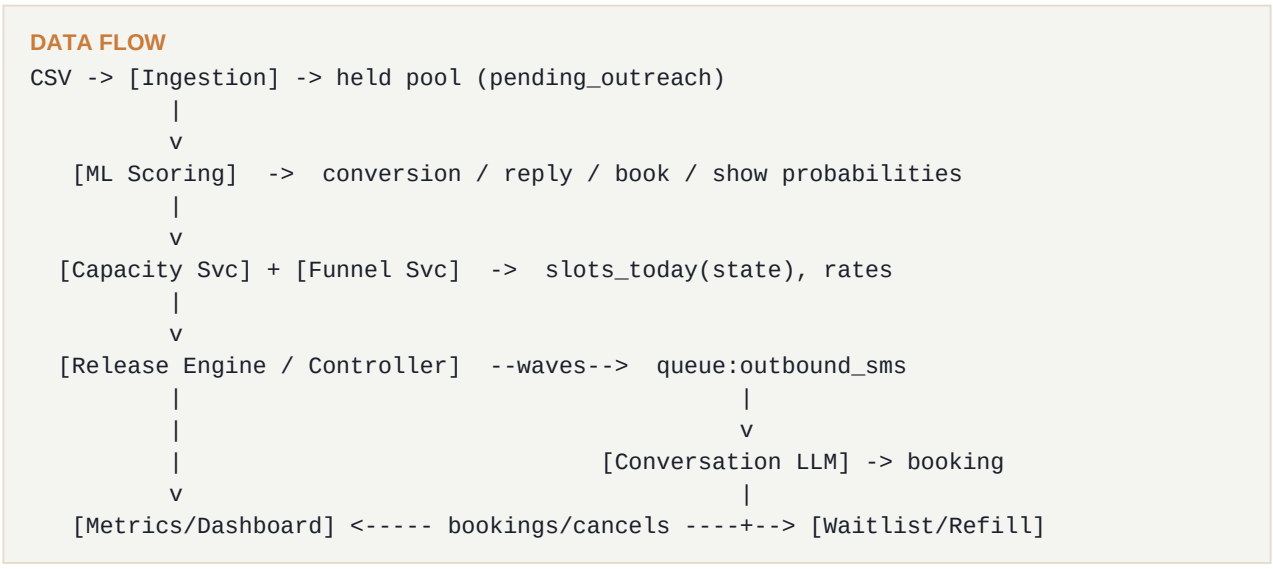
Hard ceiling: capacity, not CSV size. With N agents x S same-day slots, the day books at most N*S appointments. Surplus leads are preserved, not burned.

2. System architecture

Major components and how data flows:

- **Ingestion** - CSV import inserts leads as `pending_outreach` (held), then triggers the engine. (`app/ingestion/services/csv_import.py`)
- **ML Scoring Service** - batch-scores held leads: conversion, reply, booking, and show probabilities.
- **Capacity Service** - computes per-state open same-day slots from licensed agents.
- **Funnel Service** - maintains live reply/book/show rates (EMA from real events).
- **Release Engine (Controller)** - each cycle computes how many leads to release per state and enqueues the top-ranked ones.
- **Conversation Engine (LLM)** - the Ollama-driven SMS dialogue that books appointments (existing).
- **Booking + Assignment** - same-day slot offer, state-licensed matching, balanced agent assignment.
- **Waitlist + Refill** - parks interested-but-unslotted leads; refills cancellations.
- **Metrics/Realtime** - per-state fill %, in-flight, waste, shortfall via Socket.IO dashboard.

DATA FLOW



3. ML models & engines

Five predictive models plus one optimizer and the conversational LLM. Each model is versioned, served behind a service interface, and has a deterministic rule-based fallback so the engine still runs if a model is unavailable.

3.1 Lead Scoring model (conversion propensity)

- **Goal:** rank held leads by probability of becoming a closed deal; drives selection order and per-lead expected value.
- **Output:** `conversion_probability` in [0,1] and `lead_score` in [0,100] (fields already exist on the lead).
- **Model:** gradient-boosted trees (XGBoost / LightGBM); logistic-regression baseline.
- **Features:** state, age band, household size, income band, plan type, lead source, upload recency, time-of-day, prior engagement, historical state/source conversion rates.
- **Fallback:** rule-based score from source quality + completeness + state conversion history.

3.2 Reply-propensity model

- **Goal:** P(reply | contacted) - drives how many leads are needed to hit target replies.
- **Model:** logistic regression / GBT. **Features:** phone validity, state, time-of-day/day-of-week, source, prior contact outcomes.

3.3 Booking-conversion model

- **Goal:** P(book | reply) - the second funnel stage.
- **Features:** reply sentiment/intent (from the conversation engine), state slot availability, plan fit, response latency.

3.4 Show / no-show model

- **Goal:** P(show | booked) - used to compute a safe over-booking factor (book slightly more than slots when no-show risk is high, like airline yield management).
- **Features:** lead score, booking lead-time, time-of-day of appointment, reminder engagement, state, history.

3.5 Reply-latency model

- **Goal:** distribution of time-from-text-to-reply - sets the daily `OUTREACH_CUTOFF_HOUR` and wave timing so replies still convert same-day.
- **Model:** survival analysis (Kaplan-Meier / Cox) or an empirical per-state latency histogram.

3.6 Agent-assignment optimizer (not ML - operations research)

- **Goal:** balance bookings across agents (no overload / idle).
- **Method:** least-utilization assignment among the licensed-and-free agents for a slot; reuse `booking/services/assignment.py` capacity scoring; optional min-max load LP for batch rebalancing.

3.7 Conversation LLM (existing)

- Ollama-served model handles the actual SMS dialogue, intent detection, objection handling, and booking tool-calls. The capacity engine decides WHO and WHEN to contact; the LLM decides WHAT to say.

4. Capacity calculations

All quantities are computed per state S each control cycle. Rates are exponentially-weighted moving averages (EMA) of real events, falling back to configured defaults until enough history exists.

CORE FORMULAS (per state, per cycle)

```
slots_today(S)   = sum over agents a licensed in S of open_slots_today(a)
reply_rate(S)    = EMA( replied / contacted )            # P(reply)
book_rate(S)     = EMA( booked  / replied   )            # P(book | reply)
show_rate(S)     = EMA( shown   / booked    )            # P(show | booked)
effective_conv(S)  = reply_rate(S) * book_rate(S)        # contacted -> booked
# over-book for no-shows so SHOWN appointments fill the day:
target_bookings(S) = slots_today(S) / max(show_rate(S), SHOW_FLOOR)
leads_needed(S)  = ceil( target_bookings(S) / effective_conv(S) * (1 + BUFFER) )
in_flight(S)     = leads contacted today in S not yet resolved
release(S)       = max( 0, leads_needed(S) - in_flight(S) )
```

Precise per-lead selection (preferred over aggregate rates)

Instead of dividing by average rates, select individual leads using their own model probabilities until expected bookings cover the remaining gap. This wastes fewer leads because high-probability leads count more:

GREEDY EXPECTED-VALUE SELECTION

```
gap(S)          = target_bookings(S) - expected_bookings_in_flight(S)
for lead in held(S) sorted by score desc:
    p_book = P(reply|lead) * P(book|reply,lead)     # from models 3.2 & 3.3
    if running_sum_of_p_book >= gap(S) * (1+BUFFER): break
    release lead
```

WORKED EXAMPLE (one state, Florida)

slots_today = 200. reply_rate = 0.20, book_rate = 0.50, show_rate = 0.80.
effective_conv = 0.20 x 0.50 = 0.10.
target_bookings = 200 / 0.80 = 250 (overbook for ~20% no-shows).
leads_needed = ceil(250 / 0.10 x 1.10) = 2,750 conversations across the day.
If 1,000 are already in-flight, release = 1,750 - spread over waves, not at once.

5. The control loop (closed-loop controller)

The engine is a feedback controller (thermostat-style): the setpoint is a full calendar; the measurement is live bookings; the actuator is lead release. It runs on upload and every `PACING_CYCLE_MINUTES` until the cutoff.

CONTROLLER PSEUDOCODE

```
on_csv_upload(): score_leads(); run_cycle()      # wave 1
every PACING_CYCLE_MINUTES until OUTREACH_CUTOFF_HOUR:
    for S in states_with_held_leads:
        recompute slots_today(S), rates, in_flight(S)
        r = release(S)                            # section 4
        if utilization(S) >= TARGET_UTILIZATION: r = 0   # auto-stop
        enqueue top-r leads (by score) into queue:outbound_sms
after cutoff: stop new first-touch; keep booking + waitlist + refill
```

6. Same-day-first & future-day fallback

- Under the flag, constrain the SMS slot generator (`orchestrator._union_slots_for_agents -> generate_ny_anchored_slots`) to a TODAY-only horizon (it currently spans several business days).
- Future-day offers turn on ONLY when (a) today's capacity in S is exhausted AND (b) waitlist depth in S exceeds remaining same-day inventory. Gated by `FUTURE_DAY_FALLBACK_ENABLED`.

7. Waitlist & cancellation refill

- Interested lead, no slot -> `ai_status = awaiting_slot`; conversation + intent preserved; worked before any untouched lead.
- On cancel/no-show the slot returns to inventory; the 5-minute emergency-fill beat offers it to the highest-priority waitlisted lead, else releases fresh leads, until filled.

8. Data model & infrastructure changes

- **Lead:** use `lifecycle_stage='pending_outreach'`; add `released_at`, `wave_id`, `priority_score`; reuse `ai_status='awaiting_slot'`.
- **Events log:** contacted -> replied -> booked -> shown (already in messages / appointments / dispositions) - the training + EMA source.
- **Redis:** per-state counters (in_flight, released_today, fill%) + the existing `queue:outbound_sms`.
- **Model registry:** versioned artifacts (local/S3), loaded by the scoring service; nightly retrain job.
- **Celery beat:** new `pacing.tick` task; reuse the 5-min emergency-fill beat for refills.

9. Implementation steps (detailed, phased)

Everything ships behind `SAME_DAY_PACING_ENABLED` (default false). Early phases run in dry-run/log-only so the live pipeline is unchanged until switched on.

Phase 0 - Foundations

- Add the lead columns + lifecycle/ai_status values (Alembic migration).
- Build the event-extraction query that yields contacted/replied/booked/shown per state for rates + training.

Phase 1 - Stop the blast

- In `bulk_import_leads_from_csv`, replace the `rpush 'queue:outbound_sms'` loop with setting `pending_outreach` (flag-gated). Emit one 'import_complete' event.

Phase 2 - Capacity + Funnel services

- `app/pacing/capacity.py`: slots_today(S) via `booking_agents_for_state` + `get_available_slots_for_agent`.
- `app/pacing/funnel.py`: EMA reply/book/show rates per state, with defaults.

Phase 3 - ML scoring service

- `app/ml/lead_scoring.py` + `app/ml/propensity.py`: load models, batch-score on import; rule-based fallback.
- Offline training pipeline (separate job) producing versioned artifacts.

Phase 4 - Release engine + waves (dry-run first)

- `app/pacing/release.py`: compute release(S) (section 4), select leads (greedy EV), enqueue, mark released.
- Celery beat `pacing.tick` every `PACING_CYCLE_MINUTES`. Ship LOG-ONLY first (log what would release; no sends).

Phase 5 - Same-day-only booking offer

- Constrain slot generation to today under the flag.

Phase 6 - Waitlist + cancellation refill

- `awaiting_slot` handling; wire the emergency-fill beat to the waitlist.

Phase 7 - Agent load balancing

- Least-utilization assignment on every booking via `assignment.py`.

Phase 8 - Future-day fallback

- Enable section 6 fallback behind its flag + threshold.

Phase 9 - Dashboard + model feedback loop

- Ship section 11 metrics; nightly retrain from accumulated events; shadow-evaluate new models before promotion.

Rollout: (1) Phases 0-4 in dry-run, validate math vs real capacity; (2) enable Phase 5 + turn on for one tenant, watch fill %; (3) add Phases 6-8; (4) add Phase 9.

10. Corner cases & solutions

Situation	How the engine handles it
Uploaded list bigger than capacity	Fill the day, then auto-stop (section 5). Surplus stays `pending_outreach` for tomorrow - not burned.

Situation	How the engine handles it
Uploaded list too small to fill the day	Work all of it; raise a 'shortfall to full = K' alert from the metrics service.
State has no licensed agent	slots_today(S)=0 -> release(S)=0; leads held + admin alert.
Reply lag (replies arrive hours later)	Reply-latency model sets `OUTREACH_CUTOFF_HOUR`; new first-touch stops mid-afternoon so replies still convert same-day.
High no-show risk	Show model raises the over-booking factor (target_bookings = slots / show_rate), capped to avoid double-booking.
Cancellations / no-shows	Slot returns to inventory; emergency-fill beat refills from waitlist, then fresh leads.
Low reply rate today	EMA reply_rate drops -> leads_needed rises -> controller releases more (within send limits).
Two leads accept the same slot	Slot lock at booking; first commit wins, the other is re-offered the next slot or waitlisted.
Duplicate / opted-out (DNC) leads	Dedup at import; DNC/consent check blocks send (existing compliance guard).
Model unavailable / cold start	Deterministic rule-based scores + default funnel rates keep the engine running.
Today full + long waitlist	Future-day fallback (section 6) turns on only past the threshold.
Late-day CSV upload	Only partial same-day fill possible; dashboard surfaces it; remainder carries to next day.

11. End-to-end user flow (example)

TIMELINE - ONE BUSINESS DAY (10,000-lead CSV, 40 agents, 640 slots)

9:00 - Upload. Leads inserted as pending_outreach; ML scoring runs; engine computes per-state capacity.
9:01 - Wave 1: controller releases the release(S) count per state into the send queue.
9:00-11:00 - LLM converses + books. 200 booked. Dashboard 31% full; EMA rates update live.
11:00 - Cycle recomputes gap; small top-up wave released to cover the shortfall.
13:00 - 500 booked (78%). Show model holds a ~20% over-book buffer.
14:00 - Cancellations -> emergency-fill beat refills from the waitlist instantly.
15:30 - OUTREACH_CUTOFF_HOUR: stop new first-touch; keep booking + refilling.
17:00 - 636/640 booked (99%). ~9,000 leads preserved. Wasted-lead counter ~0.

12. Configuration & flags

Flag / setting	Purpose
SAME_DAY_PACING_ENABLED	Master switch for the engine (default false).
OUTREACH_CUTOFF_HOUR	Lead-local hour to stop new first-touch sends (from the latency model).
PACING_CYCLE_MINUTES	Controller tick interval (e.g. 15).
BUFFER (PACING_WAVE_BUFFER)	Per-wave safety overshoot (e.g. 0.10).
SHOW_FLOOR	Lower bound on show_rate to cap over-booking.
TARGET_UTILIZATION	Fill fraction that counts as 'full' (e.g. 1.0).
FUTURE_DAY_FALLBACK_ENABLED	Allow section 6 fallback.
PACING_DEFAULT_{REPLY,BOOK,SHOW}_RATE	Funnel defaults until models/EMA have data.

13. Observability & metrics

- Per state + overall: slots / booked / fill %, in-flight, leads worked vs CSV size, wasted-lead counter, shortfall-to-full, waitlist depth, live EMA rates, over-booking factor.
- Model health: prediction vs actual (reply/book/show calibration), score distribution, fallback-active flag.
- Streamed to the dashboard over the existing Socket.IO channel.

14. Upcoming roadmap - "Call Me Now" (instant agent connect)

A parallel LIVE lane next to scheduled appointments: while the AI texts a lead about booking, it also offers them a call with a licensed agent right now. If they opt in, the system finds an available licensed agent, sends a real-time accept/decline notification, and connects the call. Hot leads convert at peak intent; idle agents are filled between booked appointments. Depends on the voice stack (Sinch WebRTC softphone + call APIs) being live.

Step by step

- **1. Offer in SMS** - append an opt-in to the booking message: "Or reply NOW to talk to an agent right away." Shown only in business hours and only when a licensed agent is actually available.
- **2. Detect intent** - the conversation engine recognizes NOW / call-me / talk-now (existing LLM intent detection).
- **3. Find an agent** - `booking_agents_for_state` filtered to online+available now (the 30-second agent-presence beat already exists), picked by least-utilization.
- **4. Real-time notify** - push an Accept / Decline notification over Socket.IO with a 30-60s countdown: "Lead [name, state] wants a call NOW." On decline/timeout, cascade to the next agent.
- **5. Place the call** - on Accept, initiate via Sinch / WebRTC with recording disclosure + consent; text the lead "Connecting you now - your phone will ring."
- **6. Reserve capacity** - mark the agent busy so the engine does not also assign a booked slot at that moment; log the call-now event.

- **7. Outcome** - booked/sold -> disposition; no-answer -> fall back to booking a slot; update a separate call-now conversion metric.

Capacity-engine synergy & guardrails

- A live call consumes agent time, so it dynamically reduces that agent's same-day booked-slot capacity; the engine accounts for it in real time.
- Hottest leads (top lead_score) route to call-now when agents are free; everyone else is booked normally.
- Idle agents (no booked appointment right now) are the prime call-now targets -> pushes utilization toward 100% by filling gaps between scheduled appointments.
- Guardrails: business hours + TCPA only; offered only when a licensed agent is genuinely available; max one concurrent call-now per agent; per-lead frequency cap; opt-out honored; graceful fallback to same-day booking.
- Metrics: call-now offered / accepted / connected / converted, agent response time, abandonment - tracked separately from SMS-booking conversion.
